

STM32WLE5 LoRa Sensor Node

Hardware Design Report

Four-Layer Mixed-Signal Sensor Platform with Precision Analog Acquisition and Sub-GHz LoRa Connectivity

Design Overview

The STM32WLE5 LoRa Sensor Node is a compact mixed-signal embedded platform designed for remote sensing and low-power wireless data acquisition.

The design combines an STM32WLE5CC microcontroller with integrated Sub-GHz radio, onboard environmental and motion sensors, and a dedicated external analog path with input protection, signal conditioning, anti-alias filtering, precision voltage referencing and 14-bit ADC conversion.

The PCB uses a four-layer architecture with dedicated ground and power planes, separate analog and digital supply domains, and a controlled 50 Ω RF path to the external SMA antenna interface.

Key Design Features

- STM32WLE5CC MCU with integrated Sub-GHz / LoRa radio
- ADC141S626 14-bit external ADC with REF3033 reference
- Protected SMA external analog sensor input
- BME280 environmental sensor and MPU-6050 IMU
- Separate +3V3 digital and +3V3A analog power domains
- BGS12SN6 RF switching with matching/filter network
- 50 Ω controlled-impedance RF routing
- Four-layer PCB with dedicated GND and power planes

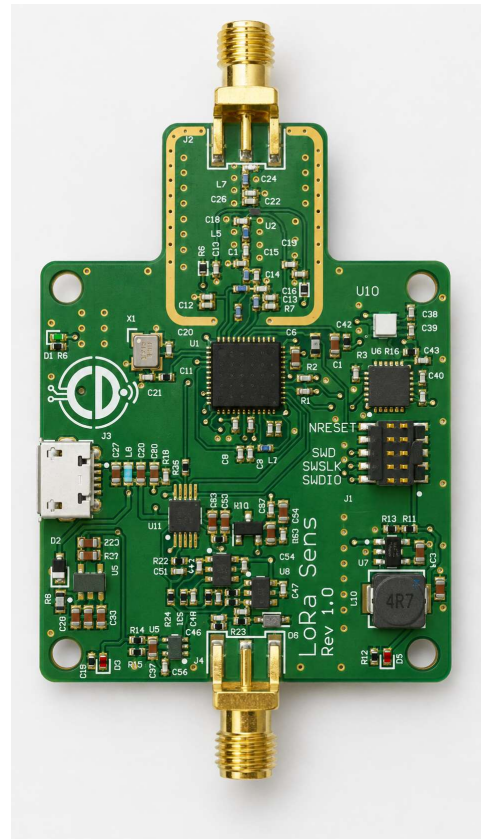


Figure 1 — STM32WLE5 LoRa Sensor Node

Designed by: Caoilte Donohoe • Document Revision: 2.0 • September 2026

1. System Overview and Design Objectives

STM32WLE5 LoRa Sensor Node — System Architecture

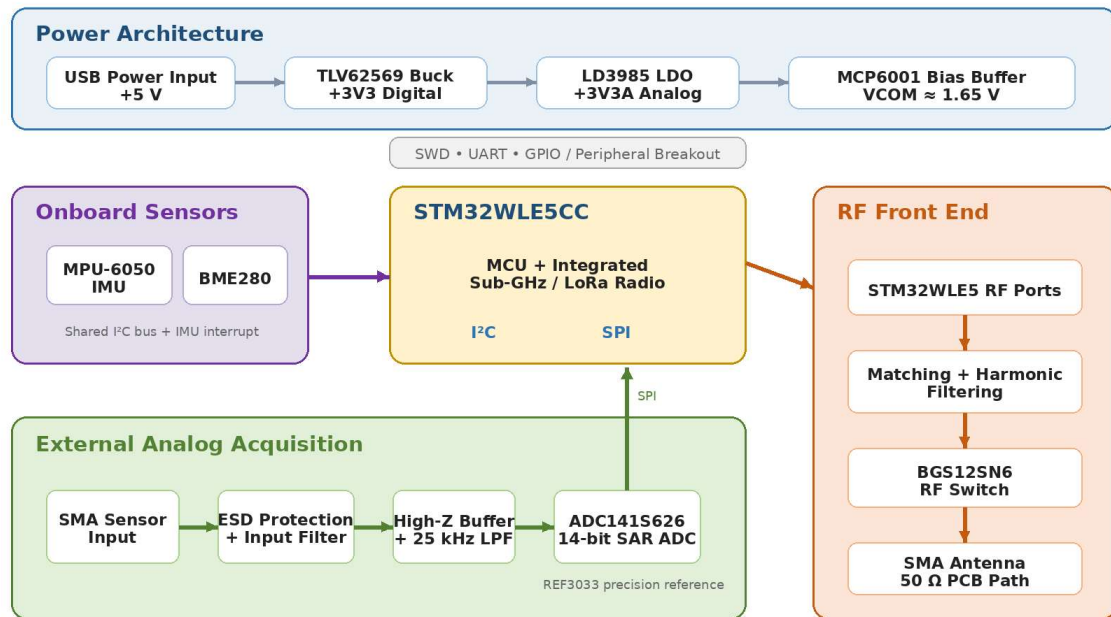


Figure 2 — STM32WLE5 LoRa Sensor Node system block diagram showing the main power, sensing, analog acquisition and RF signal paths.

1.1 Design Objective

I wanted this board to be more than a simple LoRa sensor node. The aim was to create a flexible platform that could collect useful data from sensors already fitted to the board, while also allowing an external analog sensor to be connected when a different measurement was required.

For the onboard sensing I included environmental and motion sensors. These provide temperature, humidity, pressure and movement information without needing any external hardware.

I also added a separate analog input through an SMA connector. This gives the board more flexibility and could be used, with suitable signal conditioning, for sensors such as vibration sensors, microphones, pressure sensors or other low-voltage analog outputs.

Another important part of the design was keeping the analog measurement path as clean as possible. The board contains both digital switching circuitry and an RF transmitter, so I wanted to reduce the chance of this noise finding its way into the ADC. I therefore separated the analog and digital power supplies and gave the analog section its own low-noise regulator.

1.2 System Architecture

The STM32WLE5CC sits at the centre of the design and handles the sensor interfaces, analog data acquisition and wireless communication.

The BME280 and MPU-6050 are connected to the microcontroller over I²C. The MPU-6050 also provides an interrupt output so that movement events can be detected without continuously polling the device.

The external analog input follows a separate path. The incoming signal is first protected and conditioned before passing through the anti-alias filter and into the ADC141S626 14-bit ADC. A REF3033 provides a dedicated precision reference for the ADC, and the converted data is transferred to the STM32WLE5CC over SPI.

On the RF side, I use the Sub-GHz radio already built into the STM32WLE5CC. The radio connects to the external antenna through the RF matching and filtering circuitry and a BGS12SN6 RF switch.

1.3 Main Functional Areas

I divided the board into five main areas:

- Microcontroller and Sub-GHz radio
- Environmental and motion sensing
- External analog input, AFE and ADC
- Analog and digital power supplies
- RF matching, filtering and antenna interface

Keeping these areas clearly defined also helped when I moved from the schematic into the PCB layout, particularly when separating the analog, digital and RF sections of the board.

2. Power Architecture

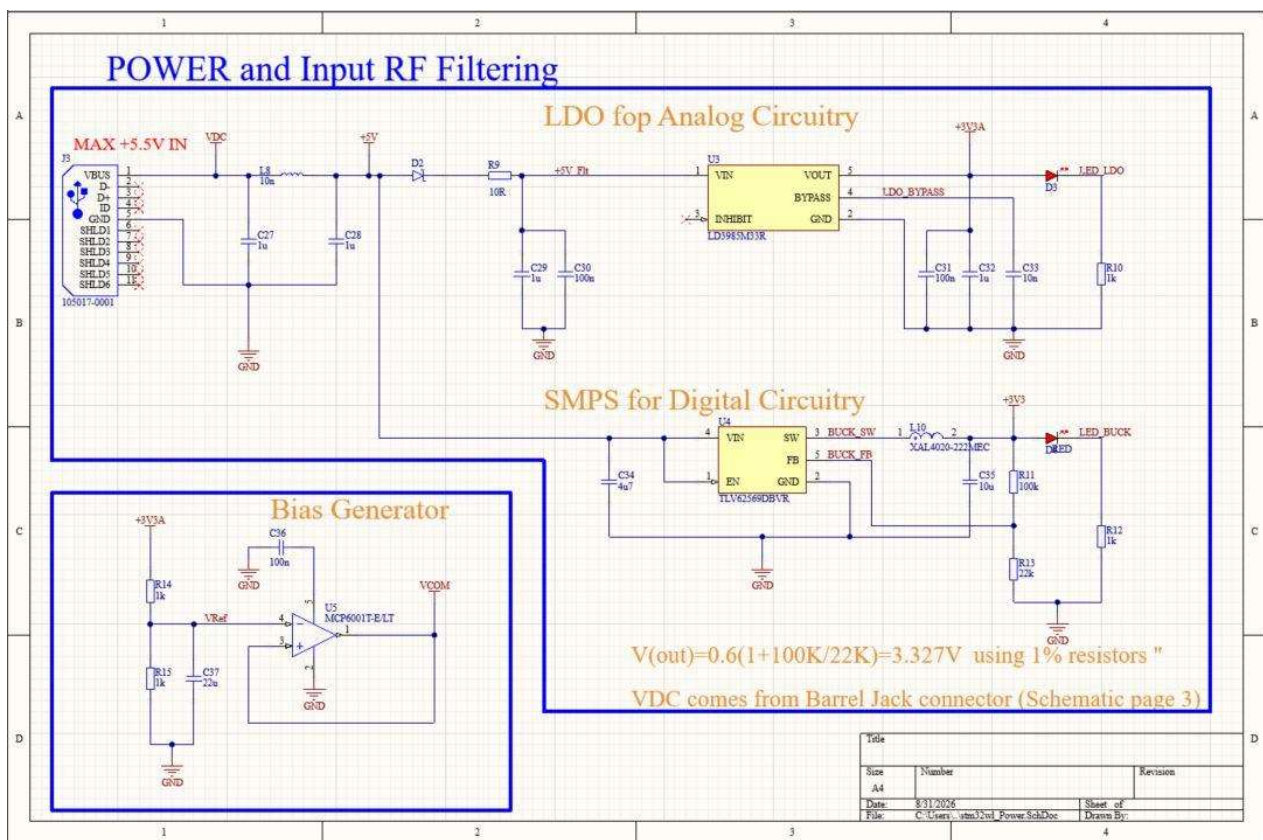


Figure 3 — Digital and analog power architecture showing the TLV62569 buck regulator, LD3985 analog LDO and MCP6001 VCOM bias generator.

2.1 Power Supply Approach

Because the board contains digital circuitry, sensitive analog circuitry and an RF section, I did not want to run everything directly from the same regulated supply.

For the main digital rail I used a TLV62569 buck converter to generate +3.3 V. A switching regulator gives good efficiency for the STM32WLE5CC and the other digital circuitry, where a small amount of switching noise is less critical.

For the analog section I took a different approach. I used an LD3985M33R low-noise LDO to generate a separate +3.3 V rail for the ADC and analog front end. My aim here was to keep noise from the switching converter and digital circuitry away from the low-level analog signal path.

This gives the board two main 3.3 V domains:

- +3V3 — digital circuitry
- +3V3A — ADC and analog circuitry

2.2 Digital Supply

The TLV62569 is used as the main 3.3 V switching regulator. I chose a buck converter here because efficiency is more important on the digital side of the board, particularly if the design is eventually used as a remotely powered or battery-operated sensor node.

The feedback network sets the output to approximately 3.3 V, which supplies the STM32WLE5CC and the digital peripherals. The power input also includes filtering before it reaches the regulators, helping to reduce conducted noise entering the board and limit noise moving between sections.

2.3 Analog Supply

The analog circuitry is supplied from the LD3985M33R LDO. Rather than powering the ADC and analog front end directly from the switching regulator, I wanted a quieter local supply for this part of the circuit. The LDO therefore provides the separate +3V3A rail used by the analog signal chain.

This separation does not completely isolate the analog and digital sections, but it gives me better control over where switching currents and supply noise flow through the board. Decoupling is then placed locally around the analog devices to keep the supply impedance low close to the circuitry that needs it.

2.4 Analog Bias Voltage

The analog front end also requires a mid-supply bias point because the board operates from a single positive supply. I generate this VCOM level using an MCP6001 op-amp. This provides a buffered bias voltage that can be used by the analog signal path without relying on a high-impedance resistor divider alone.

VCOM is separate from the ADC precision reference. The ADC reference is provided by the REF3033 on the ADC/AFE section of the design.

3. Onboard Sensors

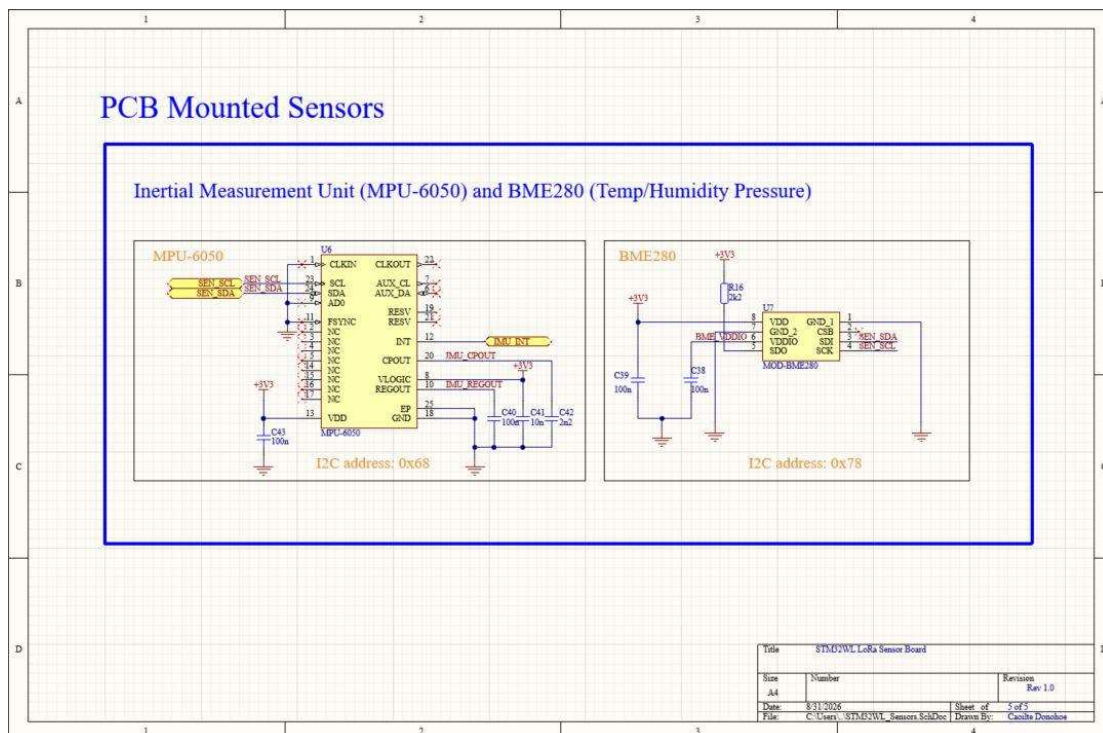


Figure 4 — BME280 environmental sensor and MPU-6050 motion sensor connected to the STM32WLE5CC through the shared I²C sensor bus.

3.1 Environmental and Motion Sensing

I wanted the board to provide useful sensing capability without relying entirely on an external sensor, so I included both environmental and motion sensing directly on the PCB.

For environmental measurements I used the BME280, which provides temperature, humidity and atmospheric pressure from a single device. For movement and orientation I used the MPU-6050, which combines a three-axis accelerometer and three-axis gyroscope.

Both sensors share the same I²C bus back to the STM32WLE5CC, which keeps the number of MCU pins required relatively low.

3.2 BME280 Environmental Sensor

I chose the BME280 because it combines three useful environmental measurements in a small digital sensor. From one device the board can measure temperature, relative humidity and atmospheric pressure.

The sensor communicates with the STM32WLE5CC over the SEN_SCL and SEN_SDA I²C lines. Using a digital sensor here also means the low-level analog measurement is handled internally by the BME280 rather than having to route additional analog signals across the PCB.

3.3 MPU-6050 Motion Sensor

The MPU-6050 provides acceleration and angular-rate information and allows the board to detect movement, vibration and changes in orientation. Like the BME280, it communicates with the STM32WLE5CC over I²C.

I also connected the MPU-6050 interrupt output to the microcontroller through the IMU_INT signal. This gives the firmware the option of responding to a motion event rather than continuously polling the sensor. That could be useful in a low-power application where the microcontroller spends much of its time in a reduced-power state and only needs to wake when something changes.

3.4 Shared Sensor Interface

Using the same I²C bus for both sensors keeps the interface simple while still allowing each device to be addressed independently. This also leaves the external analog acquisition path completely separate from the onboard digital sensors, which was important because I wanted the ADC section to remain available for sensors that provide a true analog output.

4. External Analog Input and Analog Front End

4.1 External Sensor Input

I wanted the board to have an analog input that was independent of the onboard digital sensors. This gives me the option of connecting an externally conditioned sensor without having to redesign the main board.

The signal enters through an SMA connector. I chose SMA mainly because it gives a mechanically secure connection and is convenient when working with screened cables and bench equipment.

The input is protected using a PESD3V3L1BA TVS device before the signal reaches the analog front end. This gives some protection against ESD and transient events that could otherwise be passed directly into the op-amp circuitry.

The external input is intended for appropriately conditioned, low-voltage analog signals rather than being a universal sensor interface. Depending on the external conditioning used, possible applications could include vibration, acoustic, pressure or other analog sensing.

4.2 Input Filtering and High-Impedance Buffer

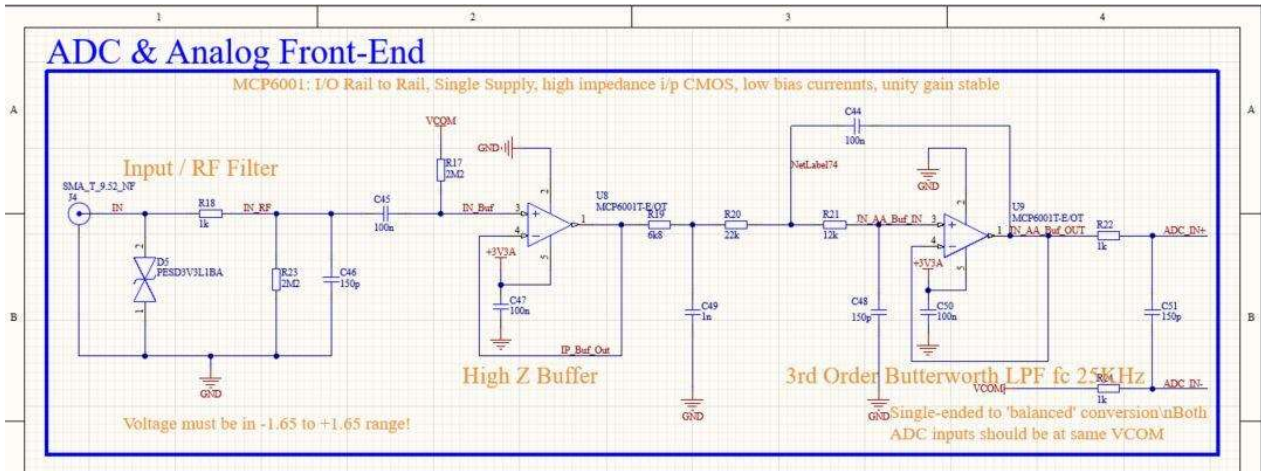


Figure 5 — External SMA analog input, ESD protection, high-impedance MCP6001 buffer and approximately 25 kHz anti-alias filter.

The first part of the analog front end provides input filtering followed by a high-impedance buffer.

I used the MCP6001 because it can operate from a single supply, has rail-to-rail input and output capability and presents a high impedance to the incoming signal. This is useful because I do not want the front end to unnecessarily load the external sensor. The buffer also gives the following filter stage a more predictable source impedance.

At the input I included filtering to reduce unwanted high-frequency content before the main anti-alias filter. This is particularly useful on a board that also contains a switching regulator and an RF transmitter.

4.3 Anti-Alias Filter

Before the signal reaches the ADC, I use a third-order low-pass filter with a cutoff region of approximately 25 kHz. I wanted a relatively flat useful passband while still getting a useful increase in attenuation above the frequencies of interest.

The third-order response is built from a first-order stage together with an active second-order Sallen-Key stage. The purpose of this filter is not just general noise reduction. It also limits the amount of high-frequency energy reaching the ADC, reducing the possibility of signals above the Nyquist frequency being aliased back into the sampled bandwidth.

The filter design and simulation are covered in more detail in Section 6.

5. ADC and Precision Reference

5.1 ADC Conversion Stage

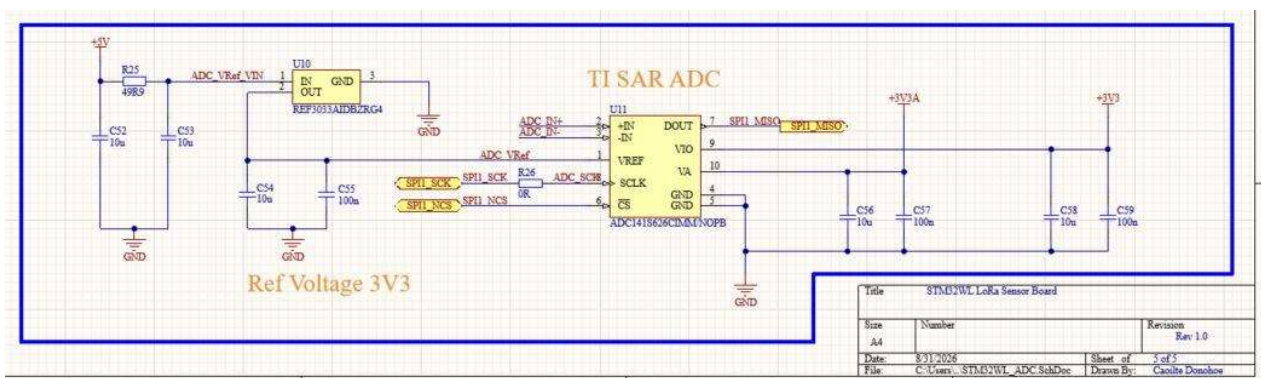


Figure 6 — ADC141S626 14-bit conversion stage, REF3033 precision reference and pseudo-differential analog input interface.

For the external analog channel I used an ADC141S626 14-bit ADC rather than relying on the ADC built into the STM32WLE5CC. This gives the analog input its own dedicated conversion stage and keeps the signal-conditioning, reference and ADC circuitry together within the analog section of the board.

The conditioned signal from the anti-alias filter is presented to the ADC through the ADC_IN+ and ADC_IN- inputs. The ADC then transfers the converted data to the STM32WLE5CC over SPI. Using SPI also keeps the analog acquisition interface separate from the I²C bus used by the onboard sensors.

5.2 Precision Voltage Reference

I used a REF3033 to provide the ADC with a dedicated 3.3 V reference. I did not want the accuracy of the conversion to depend directly on the main 3.3 V supply, particularly as that rail also powers digital circuitry. The REF3033 provides a more stable reference specifically for the ADC.

The reference is locally decoupled and powered from the analog +3V3A supply.

5.3 VCOM and Pseudo-Differential Input

Because the analog circuitry operates from a single supply, the signal needs a bias point that allows an AC waveform to move both above and below its resting level without requiring a negative supply. I generate this bias as VCOM using an MCP6001 buffer.

It is important to distinguish VCOM from the ADC reference:

- VCOM provides the bias point used by the analog signal path.
- REF3033 provides the precision reference used by the ADC for conversion.

The final ADC input is arranged as a pseudo-differential signal, with the conditioned analog signal applied to ADC_IN+ and the bias/reference side applied to ADC_IN-. This allows the ADC to measure the signal relative to the analog bias point rather than simply relative to ground.

5.4 Interface to the STM32WLE5CC

The ADC connects to the microcontroller through SPI using the clock, chip-select and data signals provided by SPI1. Once a conversion has been completed, the STM32WLE5CC can read the ADC result and combine it with measurements from the onboard environmental and motion sensors before processing or transmitting the data over LoRa.

6. Anti-Aliasing Filter Design and Simulation

For the external analog input I wanted to limit the amount of high-frequency content reaching the ADC before the signal was sampled. Rather than rely on a single RC filter, I used a third-order low-pass arrangement made up of a first-order RC stage followed by an active second-order Sallen-Key stage.

This gave me a steeper roll-off above the measurement band while keeping the circuit reasonably simple. The component values place the two filter stages in the same general region, with the RC stage at about 23.4 kHz and the Sallen-Key stage at about 25.3 kHz.

I used simulation first to check the frequency response and transient behaviour, and then repeated the analysis in Python so I could look at the same design from a signal-processing point of view.

6.1 Filter Circuit and LTspice Model

The LTspice model uses R5 and C5 as the first-order input stage, followed by the active Sallen-Key network around U1. I also included the small output RC network so that the simulation reflected the signal path presented to the ADC more closely.

For the op-amp model I used a 1 MHz gain-bandwidth product and an open-loop gain of 100 k, chosen to approximate the behaviour of the MCP6001 used in the hardware.

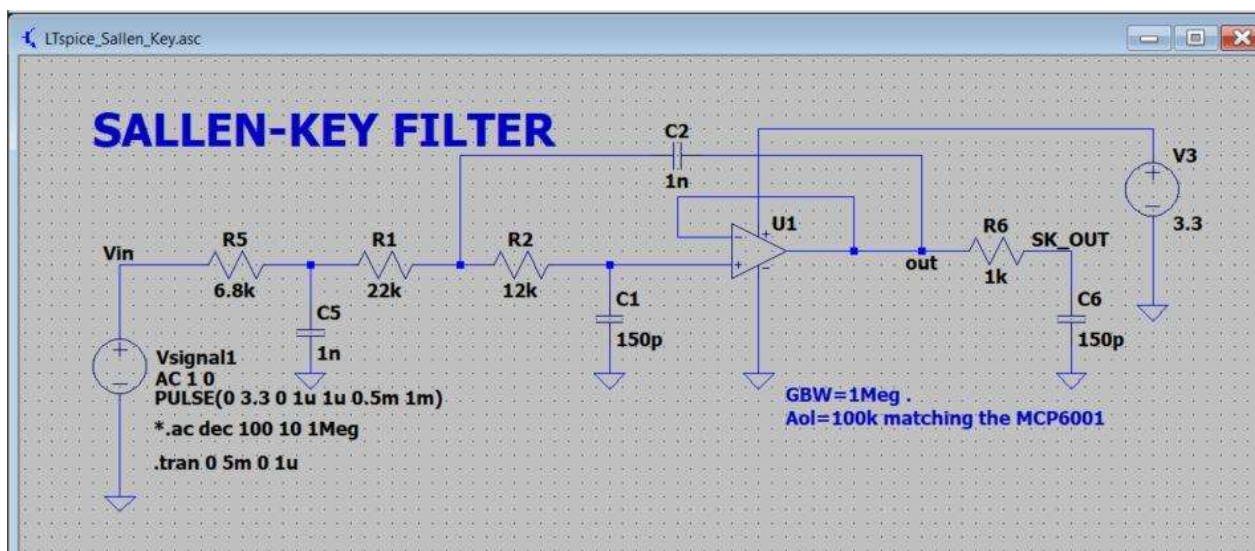


Figure 7 — LTspice model of the third-order analog low-pass filter, including the input RC stage, active Sallen-Key section and output RC network.

6.2 LTspice Frequency Response

I first checked the filter with an AC sweep. The response stays close to flat at low frequency before beginning to roll off through the 20–30 kHz region. At higher frequency the attenuation approaches the behaviour expected from a third-order low-pass network.

The LTspice cursor measurement around the transition region is useful as a practical check of the simulated cutoff, while the phase trace shows the increasing phase lag introduced as the three poles begin to dominate.

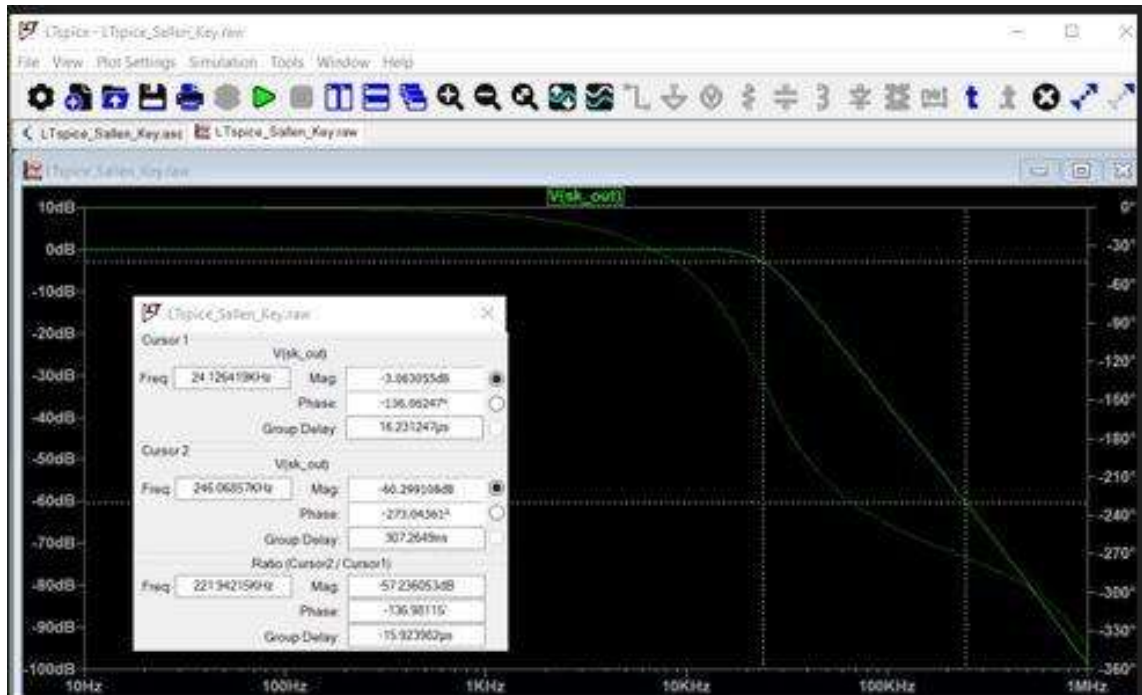


Figure 8 — LTspice AC sweep of the analog low-pass filter showing the magnitude and phase response around the transition region.

6.3 LTspice Time-Domain Response

I then moved to transient analysis. With a 10 kHz sine-wave input, the output follows the input cleanly but with the expected phase delay and a small change in amplitude. This confirmed that a signal well below the cutoff could pass through the filter without being heavily attenuated.

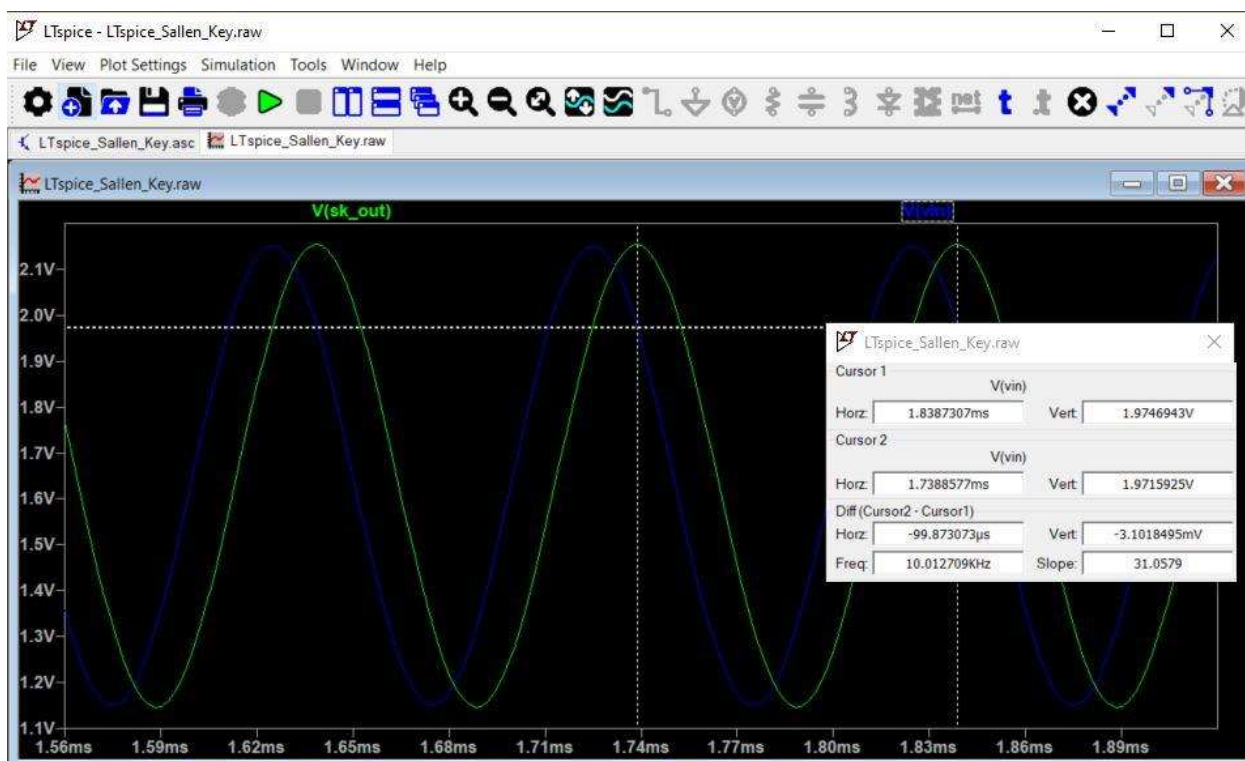


Figure 9 — LTspice 10 kHz transient response showing the input waveform and filtered output, including the expected phase delay.

A pulse test provides a more demanding check because a fast edge contains a large amount of high-frequency energy. The filter rounds the transitions and shows a small amount of overshoot and settling before reaching the steady value. This is consistent with the active second-order stage and is not a concern for the intended analog measurement path.

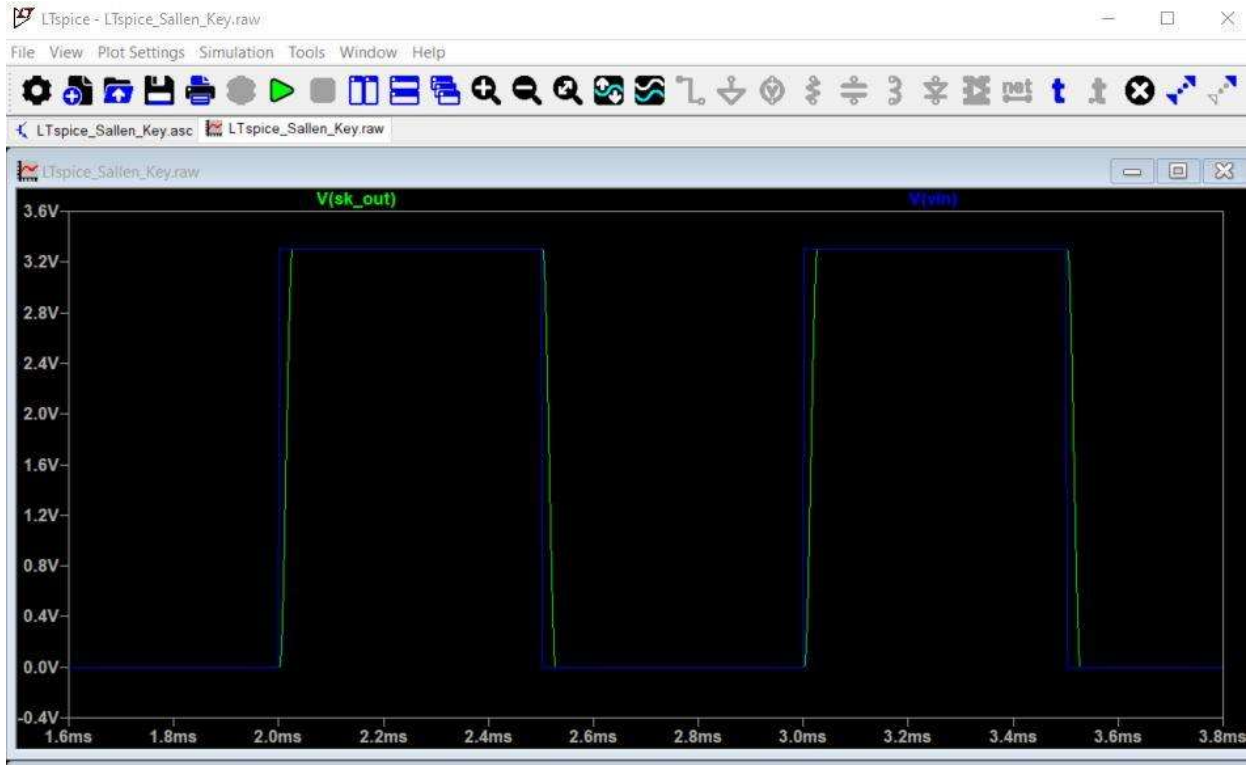


Figure 10 — LTspice pulse response showing edge smoothing, short-duration overshoot and settling produced by the analog low-pass filter.

6.4 Python Analysis Environment

After the LTspice work I repeated the main checks in Python using the Anaconda distribution and Jupyter Notebook. This made it easy to calculate the filter response, generate test signals and compare the signal before and after filtering.

Analysis tools

Anaconda — Python environment and package management

Jupyter Notebook — interactive calculation and plotting

NumPy — numerical arrays and calculations

SciPy — signal-processing and filter-analysis functions

Matplotlib — Bode, time-domain and FFT plots

6.5 Python Frequency-Domain Check

The Python calculation gave an RC cutoff of about 23.4 kHz and a Sallen-Key cutoff of about 25.3 kHz. The calculated response then rolls off toward approximately -60 dB/decade at high frequency, which is consistent with three low-pass poles.

The phase response progresses toward approximately -270° as frequency increases. This is another useful check that the complete network is behaving as a third-order filter rather than as a single low-pass stage.

RC filter cutoff frequency (Hz): 23405.1
 Sallen-Key cutoff frequency (Hz): 25291.4
 Sallen-Key Q factor: 1.234
 Roll-off slope near 10 kHz (transition region): 1.02 dB/decade
 Roll-off slope in stopband (100 kHz–1 MHz): -60.10 dB/decade

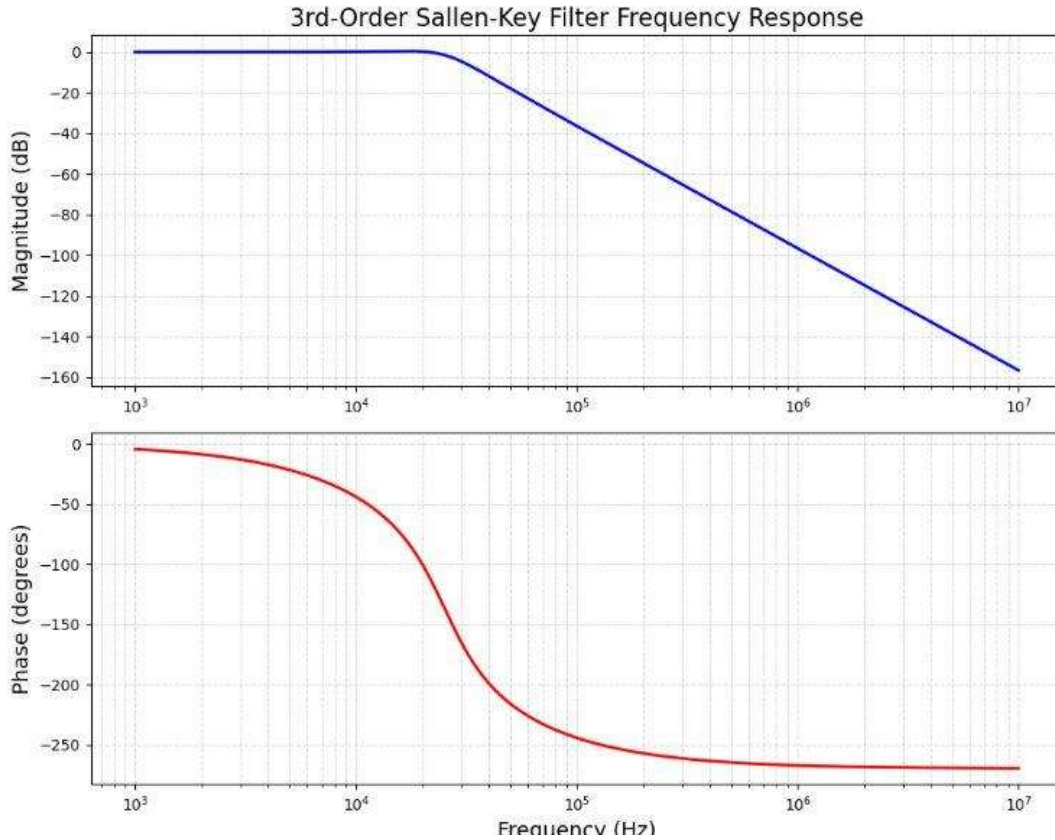


Figure 11 — Python magnitude and phase response of the third-order analog low-pass filter. The calculated transition region is centred around 25 kHz.

6.6 Python Time-Domain Check

I also generated time-domain tests in Python. The 10 kHz sine-wave result shows the same behaviour seen in LTspice: the wanted signal is retained, with a visible phase delay through the filter.

The pulse test again shows the short-duration overshoot and ringing produced by the active filter response. The important point for this design is that the waveform settles quickly and that the filter is doing what I intended—removing the high-frequency content associated with rapid transitions.

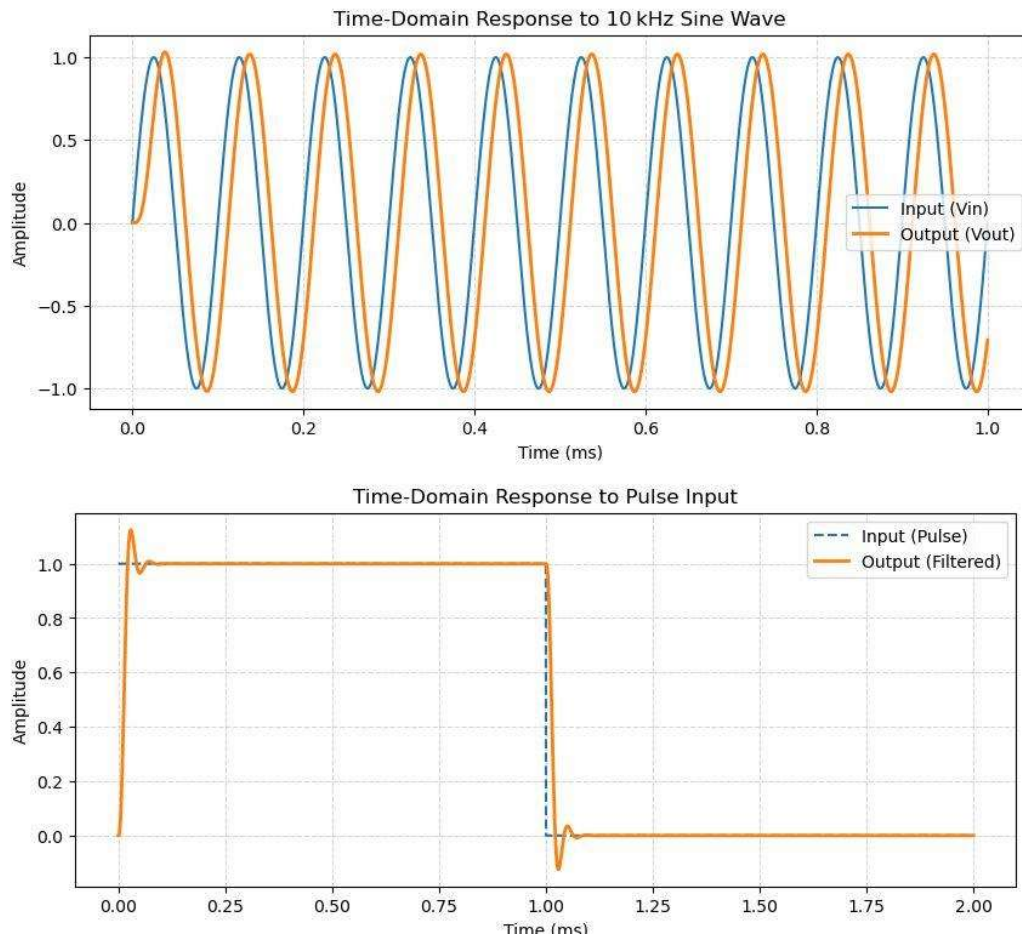


Figure 12 — Python time-domain analysis showing the 10 kHz sine-wave response and the response to a pulse input.

6.7 FFT Noise-Rejection Test

To make the filtering action easier to see, I created a test signal containing a wanted 10 kHz component together with an unwanted 100 kHz component and compared the spectra before and after filtering.

The peak at 10 kHz is the wanted signal. The second peak at 100 kHz is the deliberately added high-frequency component. After filtering, the 10 kHz component remains clearly visible while the 100 kHz peak is strongly reduced.

The deep dip between the two peaks in the unfiltered FFT should not be treated as a real notch in the hardware. It is a feature of the finite FFT calculation and the way the discrete simulated tones, FFT bins and spectral leakage combine between the two main components. For this test, the meaningful comparison is the change in amplitude at the known 10 kHz and 100 kHz frequencies.

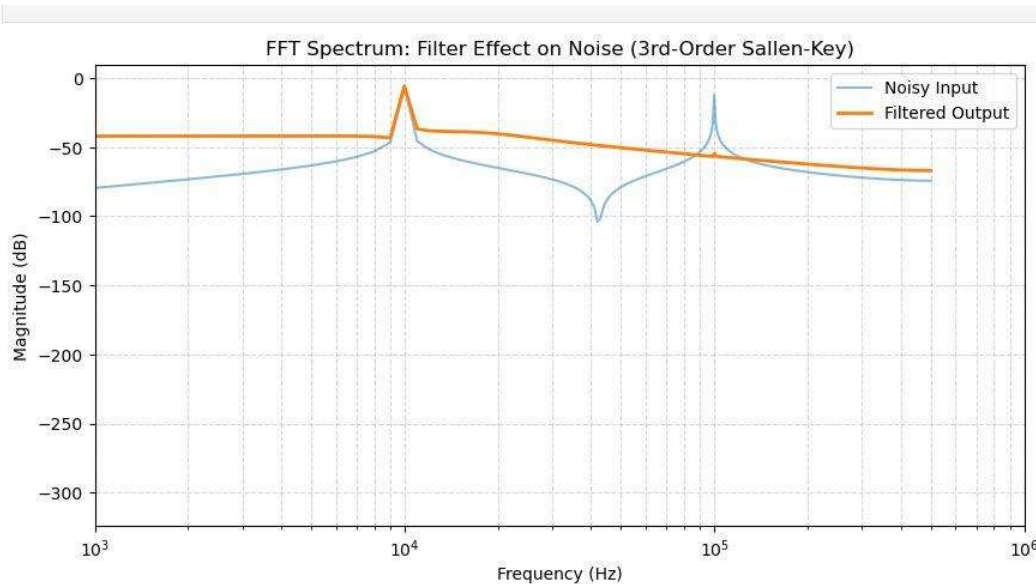


Figure 13 — FFT comparison of the noisy input and filtered output. The wanted 10 kHz component is retained while the deliberately added 100 kHz component is substantially attenuated.

6.8 Time-Domain Noise Reduction

The same test is also useful in the time domain. Before filtering, the high-frequency component is clearly visible on top of the wanted sine wave. After the filter, the waveform is much cleaner and follows the underlying 10 kHz signal.

The original Python output also produced a post-filter SNR value by comparing the filtered waveform directly with the original unshifted signal. I have not used that value in the report because the filter changes both gain and phase; without compensating for those changes, part of the wanted signal can be counted incorrectly as noise.

A more meaningful SNR calculation would first align the filtered signal in gain and phase, or calculate the wanted and unwanted power directly from the frequency-domain data.

SNR before filtering: 6.02 dB

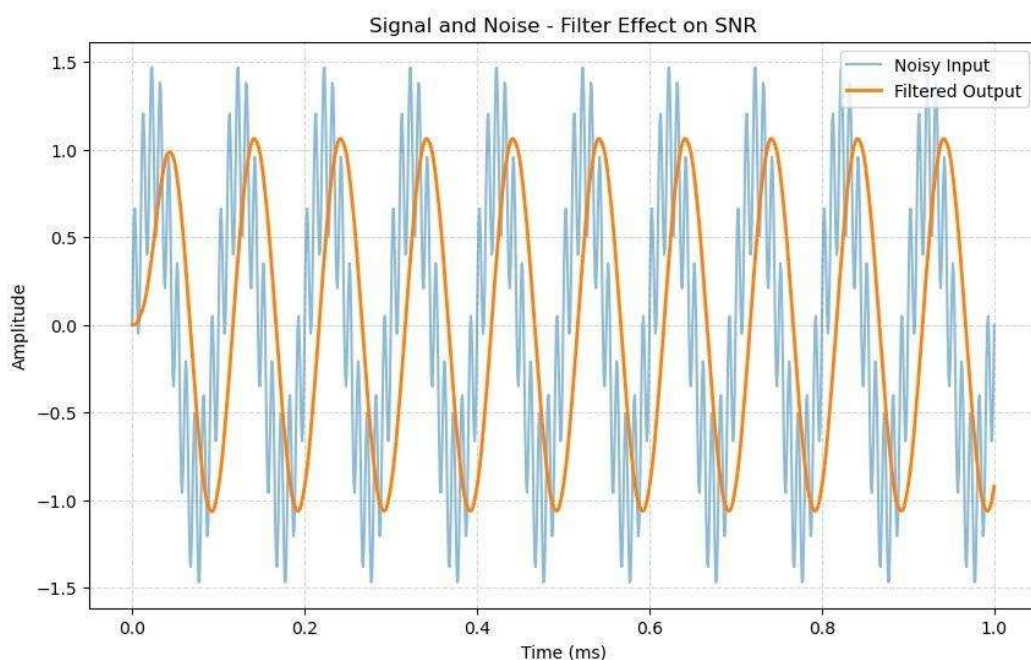


Figure 14 — Time-domain comparison of the noisy input and filtered output. The original pre-filter SNR is shown; the post-filter SNR value has been omitted because the direct calculation did not compensate for filter gain and phase.

6.9 Simulation Outcome

The LTspice and Python results gave me several independent checks on the filter before carrying it into the final analog front end.

- The transition region is centred around the intended 25 kHz range.
- A 10 kHz signal passes through with only modest attenuation and the expected phase delay.
- The high-frequency roll-off approaches the behaviour expected from a third-order low-pass network.
- The pulse response is stable, with short-duration overshoot and settling rather than sustained oscillation.
- The FFT test shows strong attenuation of the deliberately added 100 kHz component while retaining the wanted 10 kHz signal.
- The time-domain noise test gives a clear visual check that the unwanted high-frequency content is being removed.

Taken together, these results gave me enough confidence in the filter response to use the same approach in the external analog acquisition path of the STM32WLE5 sensor board.

7. RF Front-End Design and Simulation

The STM32WLE5CC already contains the Sub-GHz radio, but the RF signal still needs the correct external matching, filtering and antenna switching before it reaches the SMA connector. I treated this as a separate design area because small component values, PCB parasitics and layout all have a much greater effect at 868 MHz than they do in the lower-frequency parts of the board.

I started with a simplified LTspice model so I could understand the impedance transformation and harmonic filtering before moving to the final schematic. The simulation values shown in this section are therefore part of the RF design development; the final PCB uses the implemented component values and the BGS12SN6 RF switch shown in the current Altium design.

7.1 RF Design Approach

For this design I use the STM32WLE5 low-power RF output for the transmit path and the differential RFL_P / RFL_N inputs for receive. A single external SMA antenna is shared between transmit and receive, so the RF front end also needs a switch to route the antenna to the correct path.

The main jobs of the external RF network are to transform the STM32WLE5 RF impedance toward 50 Ω , reduce harmonic energy outside the wanted band, provide the required DC blocking and route the antenna between the transmit and receive paths.

ST's STM32WL RF matching guidance was my starting point, but I used simulation to explore the effect of the individual matching and filtering sections before laying out the final network.

7.2 Initial 868 MHz Impedance Match

For the first LTspice model I represented the STM32WL RFO_LP output at 868 MHz with a 15.27 Ω series resistance and a 144 pF series capacitance. This approximates the complex source impedance I was working from and gives a simple way of testing the impedance transformation into a 50 Ω load.

The first matching stage used an inductor of about 4.45 nH and a shunt capacitor of about 5.49 pF. The simulated response is close to 0 dB around the target frequency, with the cursor in this run showing approximately -0.87 dB at 866.7 MHz.

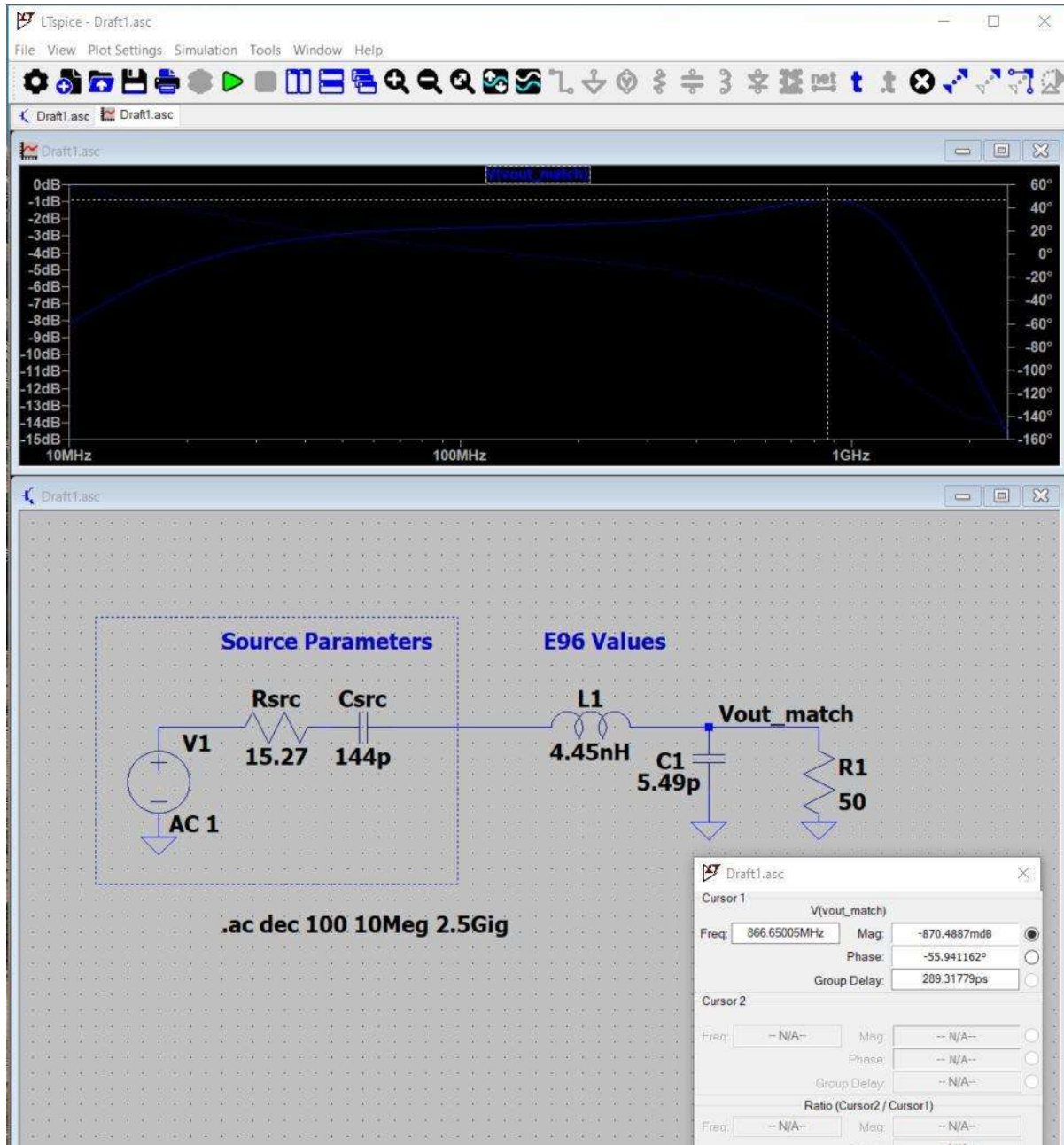


Figure 15 — Initial LTspice model of the STM32WL RF output impedance and L-match into a 50 Ω load at approximately 868 MHz.

Simulation model

This first circuit is intentionally simplified. It was useful for understanding the impedance transformation, but it does not include PCB transmission-line effects, package parasitics or the complete final RF switch and filter network.

7.3 Expanded TX Matching and Harmonic Filtering

Once the basic match was behaving as expected, I expanded the model to include the other sections I expected to need in the transmit path: the initial matching circuit, a harmonic/notch section, a DC-blocking capacitor, a π -filter and a simple RF-switch model.

For this development model I represented the RF switch as a small series resistance and inductance rather than modelling a specific device. That let me include the main insertion-loss and parasitic effects without making the early simulation unnecessarily complicated.

In the version without the additional final output π -filter, the simulation gave about -1.24 dB at 868 MHz and roughly 57–59 dB of attenuation at the second harmonic around 1.736 GHz. The simulated group delay at the wanted frequency was approximately 591 ps.

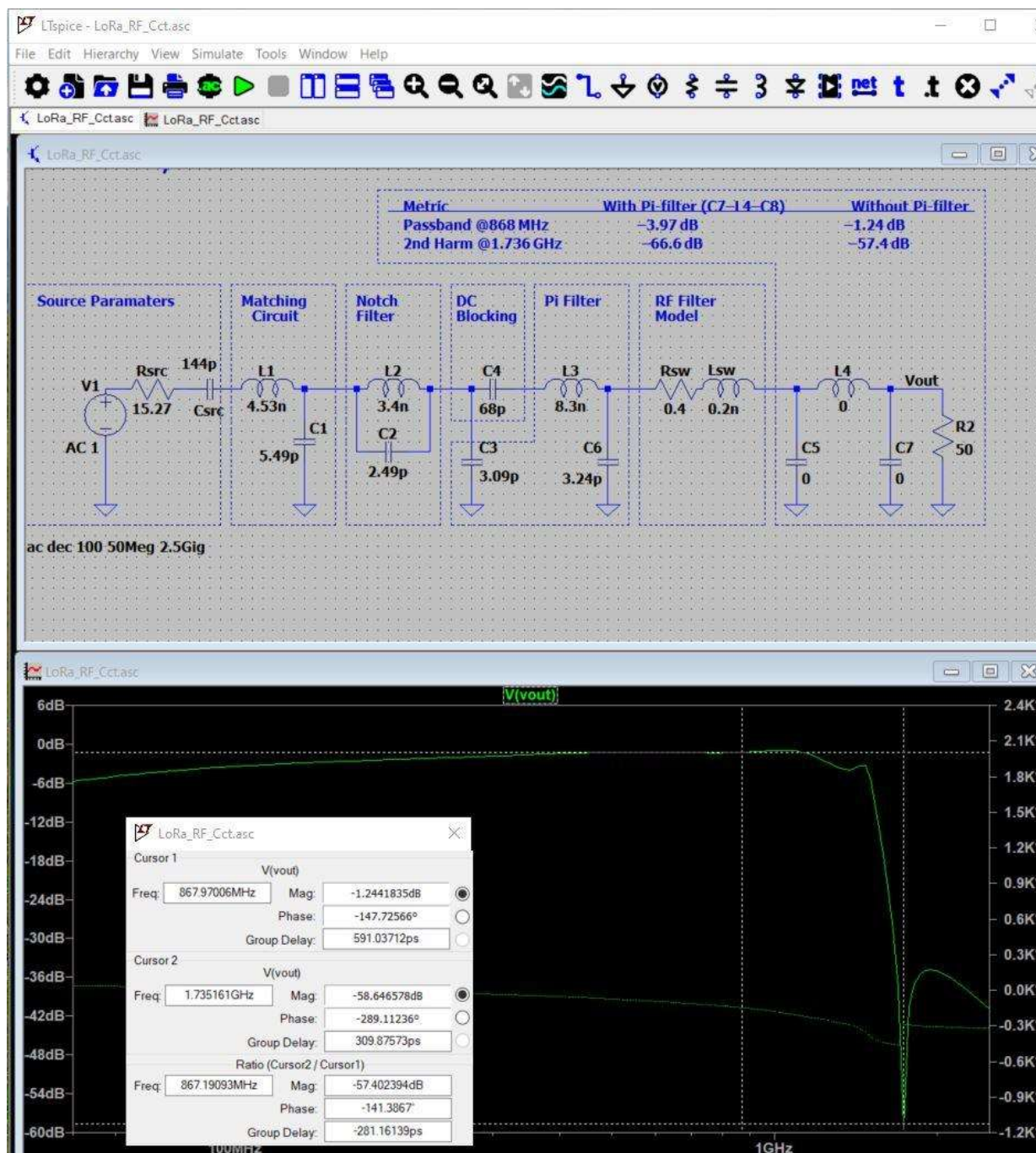


Figure 16 — Expanded LTspice transmit-path model including matching, harmonic filtering, DC blocking, π -filtering and a simplified RF-switch model.

7.4 Output π -Filter Trade-Off

I also tested an additional π -filter at the antenna side of the simulated switch. This gave better second-harmonic suppression, but it also introduced a noticeable increase in loss at the wanted 868 MHz frequency.

Simulation	Passband @ ~868 MHz	2nd harmonic @ ~1.736 GHz
Without final output π -filter	-1.24 dB	-57.4 dB
With final output π -filter	-3.97 dB	-66.6 dB

The extra stage improved the simulated second-harmonic rejection by about 9 dB, but the passband loss increased by roughly 2.7 dB. For a low-power radio link that is a significant trade-off, because every dB lost in the output network reduces the RF power available at the antenna.

This comparison was useful because it showed that adding more filtering is not automatically an improvement. The additional π -filter only makes sense if the extra harmonic suppression is actually required and the network can be tuned without giving away too much wanted-band power.

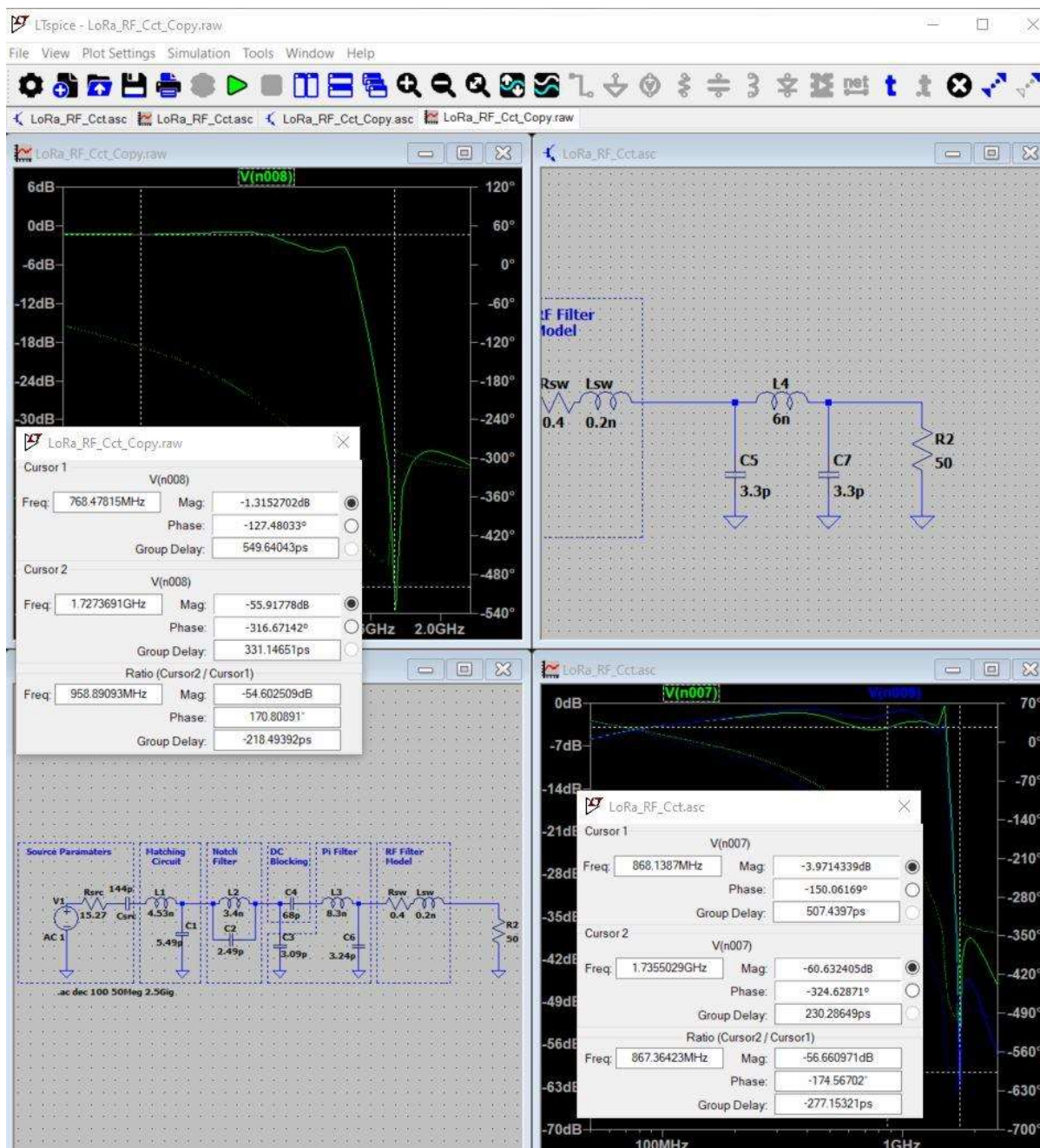


Figure 17 — LTspice comparison of the RF response with and without the additional output π -filter, showing the trade-off between harmonic rejection and passband loss.

7.5 Final Implemented RF Network

The final Altium schematic is more representative of the real hardware than the earlier LTspice development models. The simplified switch model was replaced by a BGS12SN6 RF switch, controlled from the STM32WLE5 through the FE_CTRL2 signal, so the same SMA antenna can be used for both transmit and receive.

The implemented transmit path comes from RFO_LP and uses revised matching and filter values. Examples from the final schematic include $L2 = 4.4 \text{ nH}$, $L3 = 3.4 \text{ nH}$, $C3 = 3.5 \text{ pF}$, $C4 = 68 \text{ pF}$ and $C5/C7 = 5.5 \text{ pF}$. The receive path uses the differential RFI_P and RFI_N inputs, with components including $L6 = 12 \text{ nH}$ and 4.5 pF capacitors around the receive matching section.

These values should be treated as the implemented schematic values rather than a direct copy of the LTspice values above. The simulation work helped me understand the matching and filtering behaviour, while the final component network reflects the actual STM32WLE5/BGS12SN6 implementation.

Development values versus final BOM

Figures 15–17 document the RF design process. They are not intended to be a component-for-component representation of the final PCB. The final schematic and PCB remain the source of truth for fitted RF values.

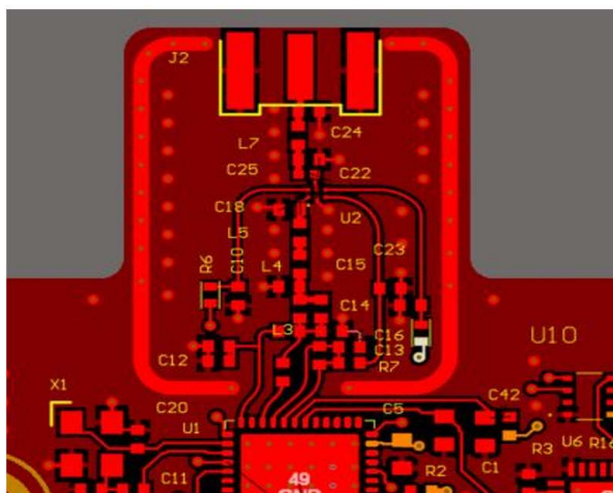
7.6 RF PCB Implementation

I placed the RF section in the narrow upper part of the PCB, directly below the antenna SMA connector. This keeps the matching and switching components grouped together and avoids routing the antenna path through unrelated digital or analog circuitry.

The RF nets use a dedicated 50Ω routing rule in Altium. I also kept a continuous ground reference around the RF area and added a line of grounded vias around the perimeter. The exposed grounded copper boundary provides a clear physical separation around the RF section and ties the surrounding top-side ground back to the internal ground structure.

The layout view and PCB below show the same RF area from two different perspectives. The signal path is kept compact, the small RF components are clustered close to the SMA connection, and the via fence follows the outside of the RF region.

(a) PCB layout



(b) Manufactured-style render



Figure 18 — Final RF PCB implementation: (a) Altium layout of the RF section and (b) PCB showing the SMA connector, RF component placement, grounded boundary and via fencing.

7.7 RF Verification Status

The LTspice work gave me a useful circuit-level check of the matching and harmonic-filtering approach, but it is not a substitute for RF measurement on the manufactured board. At these frequencies the final response will also depend on component tolerances, package parasitics, the PCB dielectric stack-up and the exact transmission-line geometry.

For hardware verification I would check the antenna port with a VNA and then measure conducted transmit power and harmonic content with suitable RF test equipment. Those measurements would allow the matching network to be tuned if required and would confirm the real insertion loss and harmonic performance of the completed PCB.

7.8 Section Summary

The RF work progressed from a simple 868 MHz impedance model, through a more complete matching and harmonic-filter simulation, and finally into the implemented STM32WLE5/BGS12SN6 front end and controlled-impedance PCB layout.

The main lesson from the simulation was that harmonic suppression and passband loss have to be considered together. That trade-off influenced the final approach and helped keep the RF path focused on the filtering that was useful rather than simply adding extra stages.